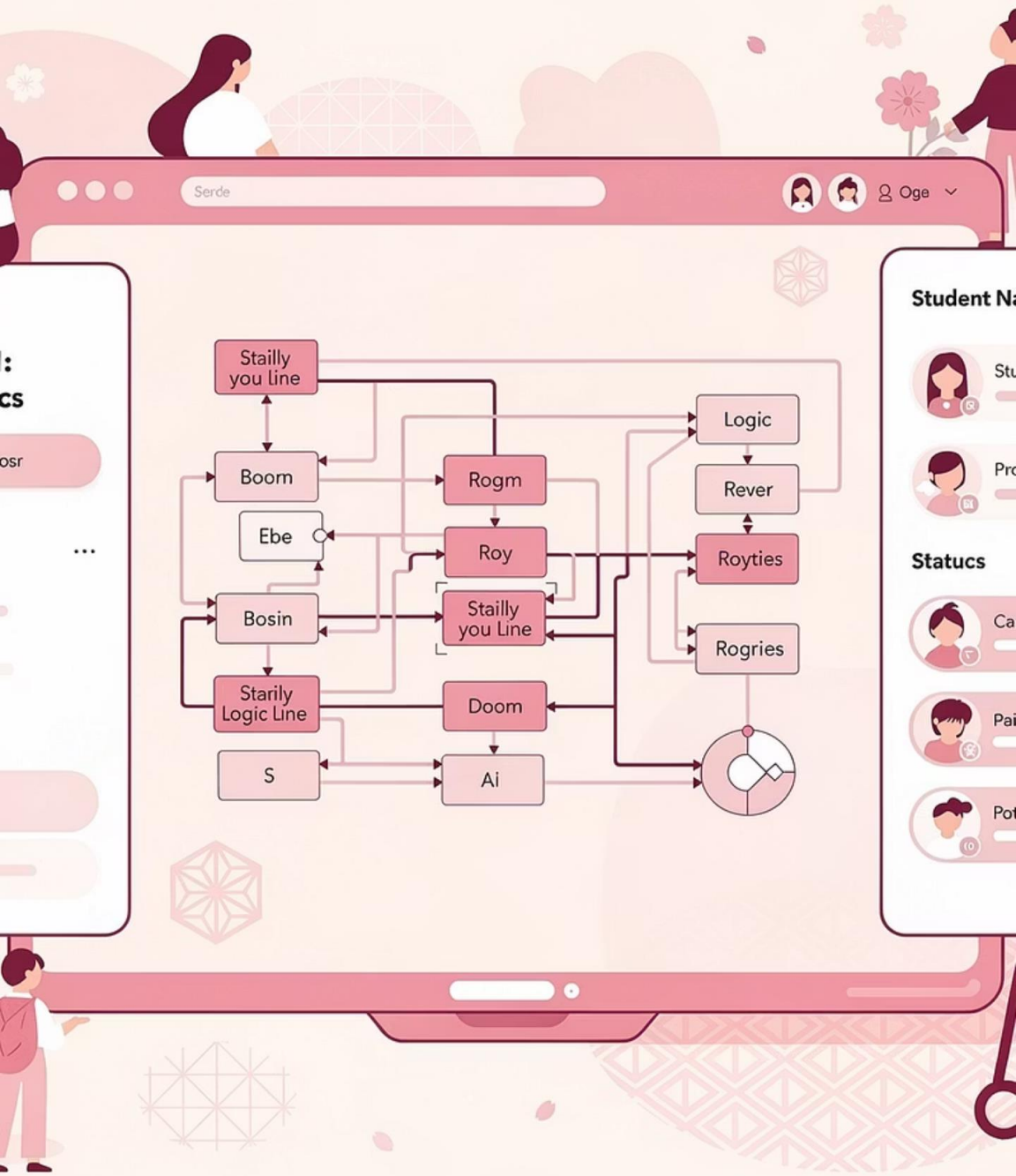




Sanal Mantık Devresi Simülatörü

Ders Kodu: BLM101 - Bilgisayar Mühendisliğine Giriş

Öğrenci Adı: Elif ÇİN

Öğrenci Numarası: 25360859042

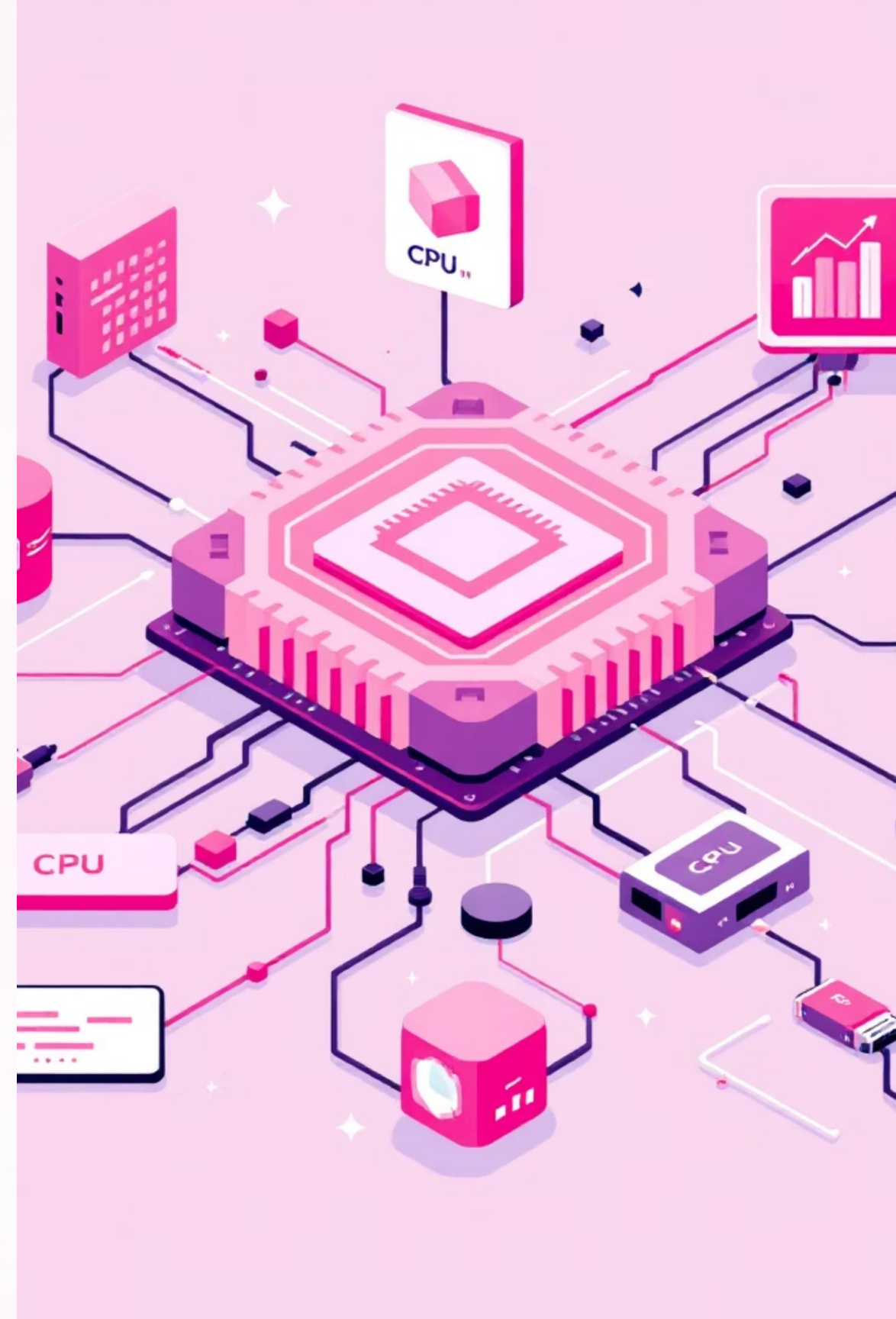
GİRİŞ

Bilgisayar Mimarisine Genel Bakış

Bilgisayar mimarisi, bir bilgisayar sisteminin tasarımı, organizasyonu ve işlevsel davranışını tanımlayan temel yapıdır. Modern bilgisayarlar, John von Neumann tarafından 1945 yılında önerilen ve bugün hala kullanılan temel mimari prensiplere dayanır.

Bu mimari, beş temel bileşenden oluşur: merkezi işlem birimi (CPU), bellek birimi, giriş birimleri, çıkış birimleri ve bunları birbirine bağlayan veri yolları. Her bileşen, bilgisayarın genel işleyişinde kritik bir rol oynar ve birbirleriyle sürekli etkileşim halindedir.

Bilgisayar mimarisini anlamak, yazılım ve donanım arasındaki ilişkiyi kavramak için esastır. Bu temel bilgi, veri manipülasyonu ve mantık devrelerinin nasıl çalıştığını anlamak için sağlam bir zemin oluşturur.



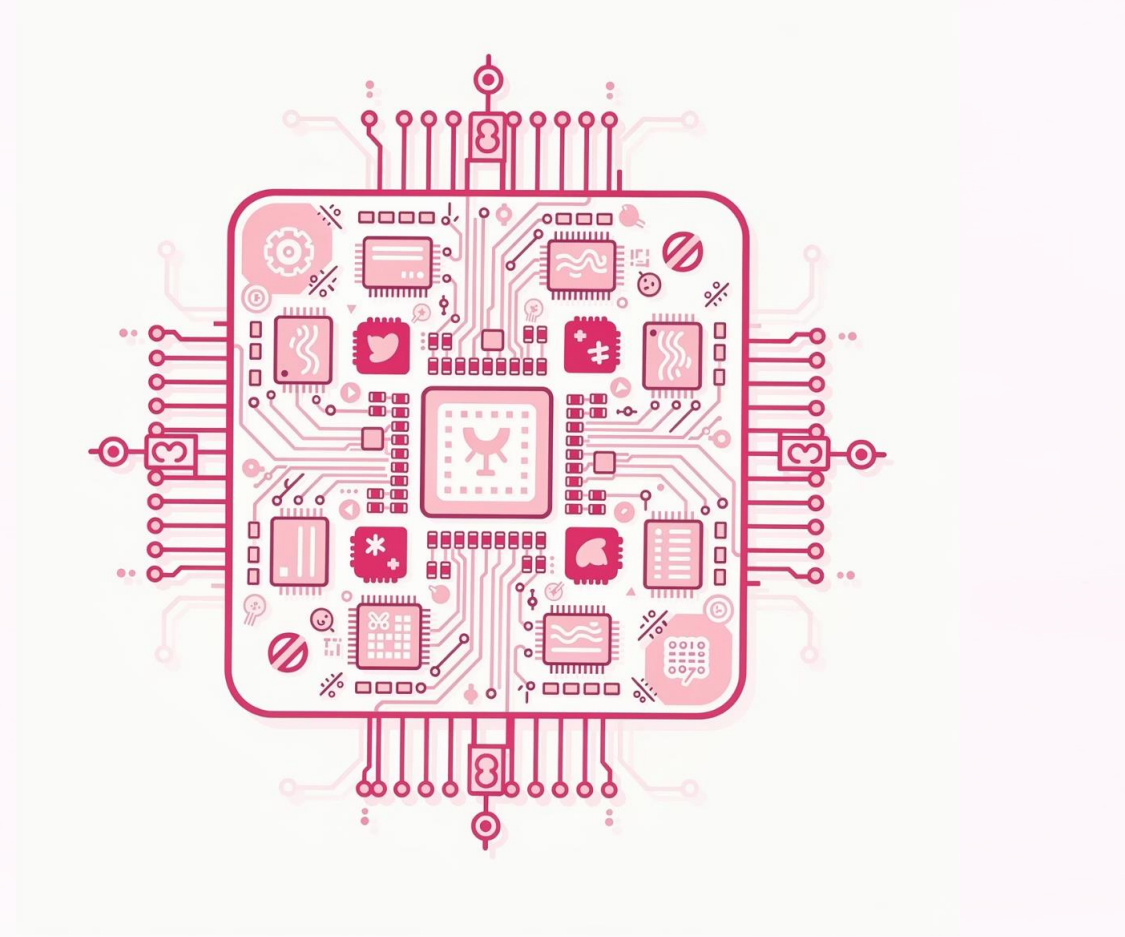
Merkezi İşlem Birimi (CPU)

Merkezi İşlem Birimi (CPU), bilgisayarın "beyni" olarak adlandırılır ve tüm hesaplama işlemlerini yürütür. CPU, programlardaki talimatları yorumlar, aritmetik ve mantıksal işlemleri gerçekleştirir ve sistem kaynaklarını yönetir.

CPU'nun temel görevleri şunlardır:

- Bellekten talimatları getirme (fetch)
- Talimatları çözümleme (decode)
- Talimatları yürütme (execute)
- Sonuçları geri yazma (write-back)

Modern CPU'lar saniyede milyarlarca işlem gerçekleştirebilir ve çok çekirdekli yapılarıyla paralel hesaplama kapasitesine sahiptir. CPU'nun performansı, bilgisayarın genel hızını doğrudan etkiler.



📌 **Önemli Not:** CPU'nun hızı, saat frekansı (GHz) ve çekirdek sayısı ile ölçülür.

CPU Performansını Anlamak

Bir Merkezi İşlem Birimi'nin (CPU) performansı, çeşitli temel metriklerle ölçülür. Bu metrikler, bir işlemcinin görevleri ne kadar hızlı ve verimli bir şekilde yerine getirebileceğini gösterir.

4.5

Saat Hızı (GHz)

İşlemcinin saniyede gerçekleştirebileceği döngü sayısı. Yüksek değer, daha hızlı işlem anlamına gelir.

8

Çekirdek Sayısı

Bir CPU'daki bağımsız işlem birimlerinin sayısı. Çoklu çekirdekler, aynı anda daha fazla görevi işleyebilir.

16

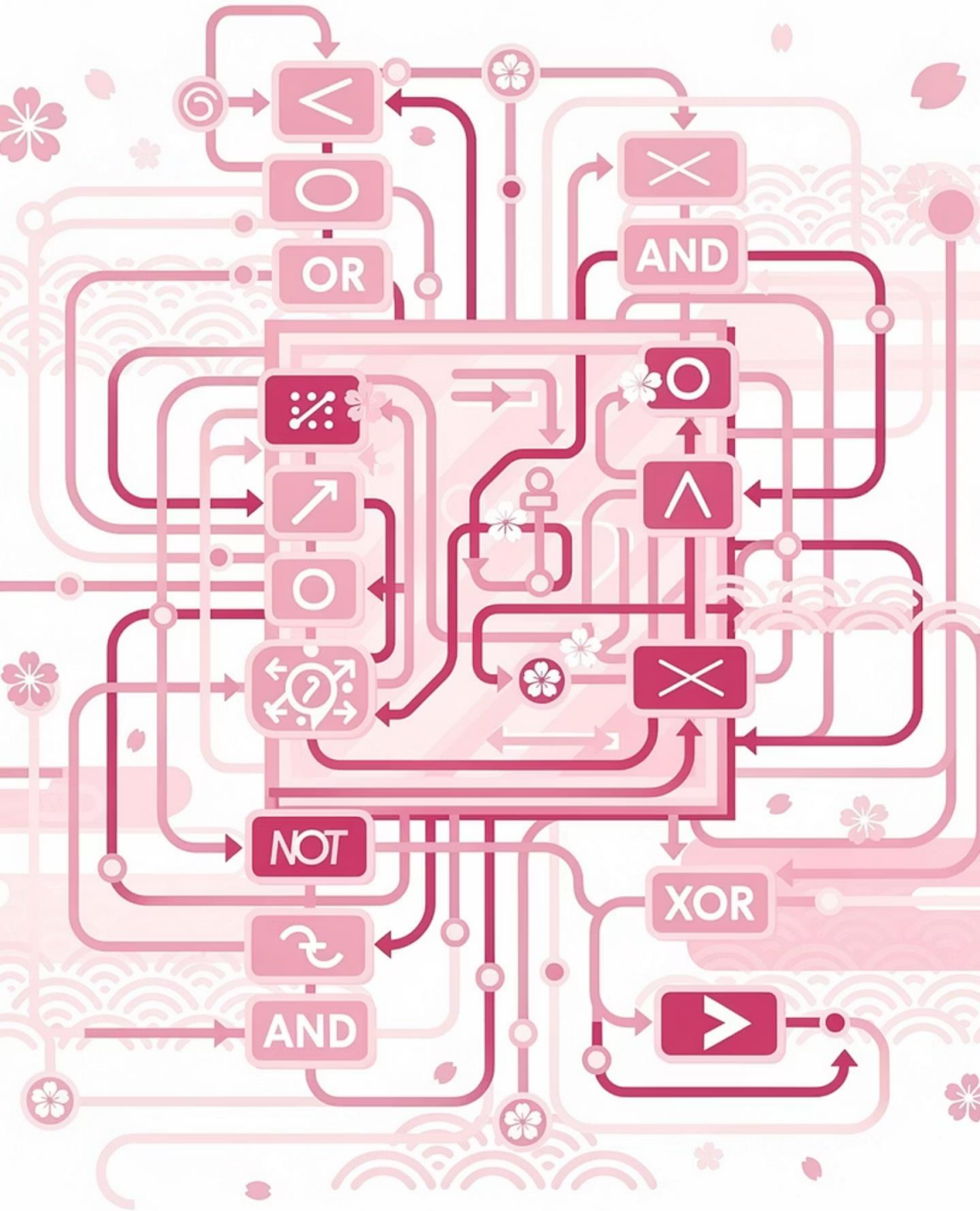
İş Parçacığı Sayısı

Hyper-threading gibi teknolojilerle her çekirdeğin işleyebileceği eşzamanlı görev sayısı.

32

Önbellek Boyutu (MB)

CPU'nun sık erişilen verileri hızlı bir şekilde depoladığı dahili hafıza. Daha büyük önbellek, performansı artırır.



ALU

Aritmetik ve Mantık Birimi (ALU)

Aritmetik ve Mantık Birimi (ALU), CPU'nun en kritik bileşenlerinden biridir ve tüm matematiksel hesaplamaları ve mantıksal karşılaştırmaları gerçekleştirir. ALU, dijital devrelerin kalbi olarak kabul edilir ve mantık kapılarından oluşan karmaşık bir yapıya sahiptir.

Aritmetik İşlemler

- Toplama ve çıkarma
- Çarpma ve bölme
- Artırma ve azaltma

Mantıksal İşlemler

- AND, OR, NOT operasyonları
- Bit kaydırma işlemleri
- Karşılaştırma işlemleri

Veri Transferi

- Kayıtlar arası veri taşıma
- Bellek okuma/yazma
- Durum bayraklarını güncelleme

ALU'nun tasarımı tamamen mantık kapılarına dayanır ve bu nedenle mantık devrelerini anlamak, bilgisayar mimarisinin temelini kavramak için kritik önem taşır.

Terimler Sözlüğü

Mantık Kapısı

Dijital devrelerde bir veya daha fazla ikili girişi işleyerek tek bir ikili çıkış üreten temel elektronik bileşen.

İkili Sayı Sistemi

Bilgisayarların veriyi temsil etmek ve işlemek için kullandığı, yalnızca 0 (kapalı) ve 1 (açık) değerlerini içeren sayı sistemi.

Boolean Cebiri

Mantıksal operasyonları ve ikili değişkenleri inceleyen, dijital devrelerin tasarımında ve analizinde kullanılan matematiksel bir sistem.

Bu temel terimler, dijital elektronik ve bilgisayar mimarisinin yapı taşlarını anlamak için kritik öneme sahiptir.

Devre Optimizasyonu

Dijital devre tasarımı, mantık kapılarının ve bileşenlerin düzenlenmesi, istenen işlevi yerine getirirken aynı zamanda devrenin performansını artırmak, güç tüketimini azaltmak ve fiziksel boyutunu küçültmek amacıyla optimize edilir. Bu süreç, Boolean Cebiri ve diğer mantıksal araçlar kullanılarak gerçekleştirilir.

Hız ve Performans

Daha hızlı veri işleme ve daha düşük gecikme süreleri elde etmek için kritik yol analizleri ve paralel işlem teknikleri kullanılır.

Güç Tüketimi

Taşınabilir cihazlar ve enerji verimli sistemler için düşük güç tüketimi sağlayan mantık kapıları ve tasarım yaklaşımları tercih edilir.

Alan ve Maliyet

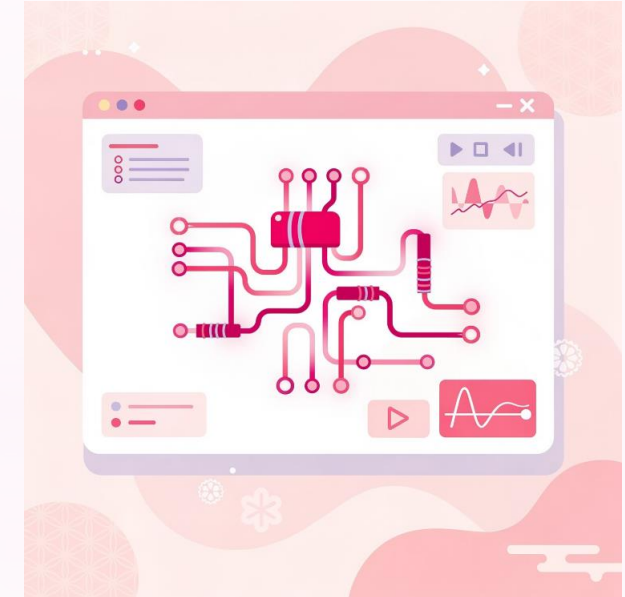
Daha az mantık kapısı kullanarak devrenin fiziksel alanını ve üretim maliyetini düşürmek, özellikle entegre devrelerde önemlidir.

Devre optimizasyonu, bilgisayar mimarisi ve dijital elektronik mühendisliğinin temel bir yönüdür ve daha verimli, güvenilir sistemler oluşturulmasında kritik bir rol oynar.

Hata Ayıklama

Hata Ayıklama (Debugging), dijital devrelerin ve yazılımların doğru çalışmasını sağlamak için hata ve tutarsızlıkları bulma ve düzeltme sürecidir. Mantık devreleri bağlamında, beklenen çıkışı üretmeyen yanlış bağlantıları veya kapı yapılandırmalarını belirlemeyi içerir.

Sanal simülatörler, bu hataları adım adım izleyerek, giriş-çıkış değerlerini analiz ederek ve devrenin her noktasındaki sinyal akışını görselleştirerek gidermede kritik rol oynar.

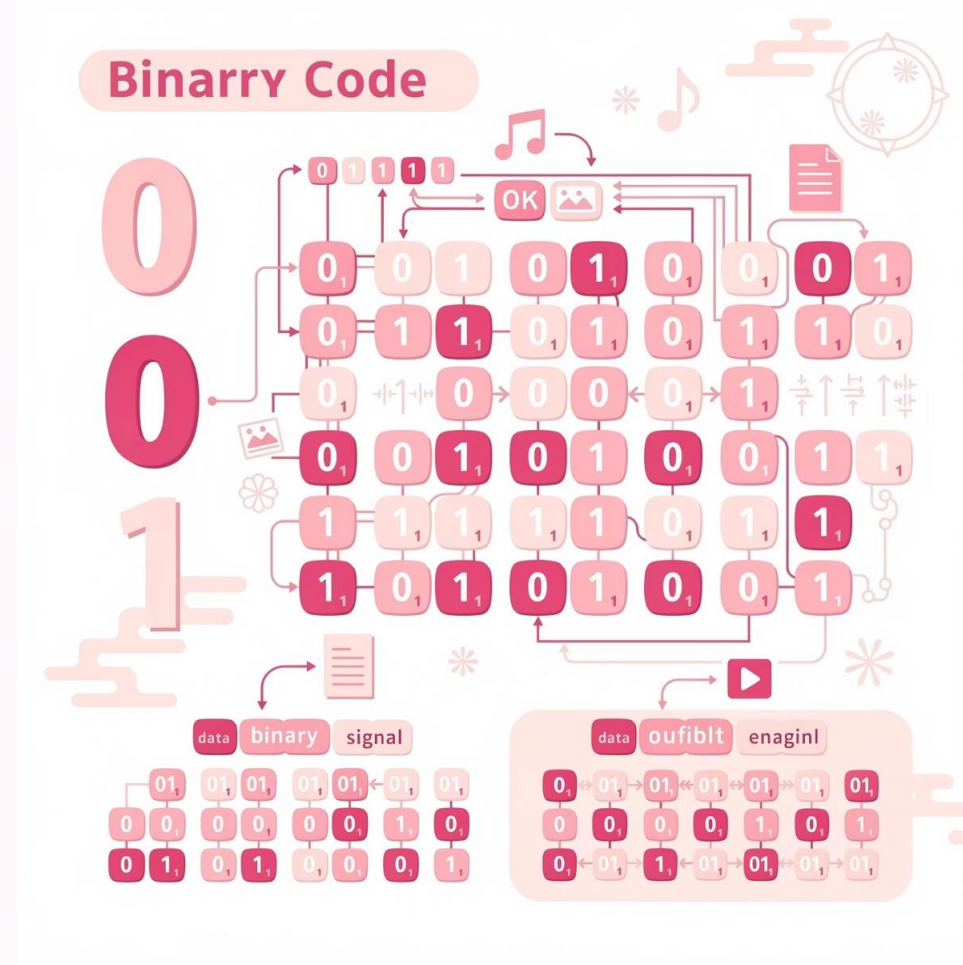


Makine Dili ve İkili Kod Sistemi

Makine dili, bilgisayarların doğrudan anlayıp işleyebildiği en düşük seviye programlama dilidir. Tamamen 0 ve 1'lerden oluşan ikili (binary) kod sistemini kullanır. Her komut, işlemci tarafından doğrudan yürütülebilen bir bit dizisi olarak temsil edilir.

İkili sistemin temel özellikleri:

- Sadece iki basamak kullanır: 0 ve 1
- Her bit bir elektrik sinyalini temsil eder (kapalı/açık)
- 8 bit = 1 byte veri birimini oluşturur
- Tüm veri türleri ikili koda dönüştürülür



❏ **Örnek:** Ondalık sayı 13, ikili sistemde 1101 olarak gösterilir: $(1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0)$

Yüksek seviye programlama dilleri (Python, Java, C++) sonunda makine diline çevrilir. Bu dönüşüm, derleyiciler veya yorumlayıcılar tarafından gerçekleştirilir ve yazılan kodun donanım tarafından anlaşılmasını sağlar.

Mantık Kapılarına Giriş

Mantık kapıları, dijital devrelerin en temel yapı taşlarıdır. Bu elektronik devre elemanları, bir veya daha fazla ikili girişi (0 veya 1) alarak Boolean mantığına göre bir ikili çıkış üretirler. George Boole'un 19. yüzyılda geliştirdiği Boolean cebri, mantık kapılarının matematiksel temelini oluşturur.

1

Dijital Sistemlerin Temeli

Tüm dijital devreler ve bilgisayar bileşenleri mantık kapılarından oluşur. Mikroişlemciler, bellekler ve ALU'lar milyonlarca mantık kapısı içerir.

2

İkili Mantık Prensibi

Her kapı, elektrik sinyallerini işler: yüksek voltaj (1/TRUE) ve düşük voltaj (0/FALSE). Bu basit iki durumlu sistem, karmaşık hesaplamaları mümkün kılar.

3

Kombinasyonel Devreler

Mantık kapıları birleştirilerek karmaşık işlemler yapan devreler oluşturulur. Toplayıcılar, çarpıcılar ve kodlayıcılar bu şekilde tasarlanır.

Mantık kapılarını anlamak, bilgisayar bilimlerinde uzmanlaşmak isteyen her öğrenci için vazgeçilmezdir. Bu temel bilgi, donanım tasarımından yazılım optimizasyonuna kadar geniş bir yelpazede uygulanır.

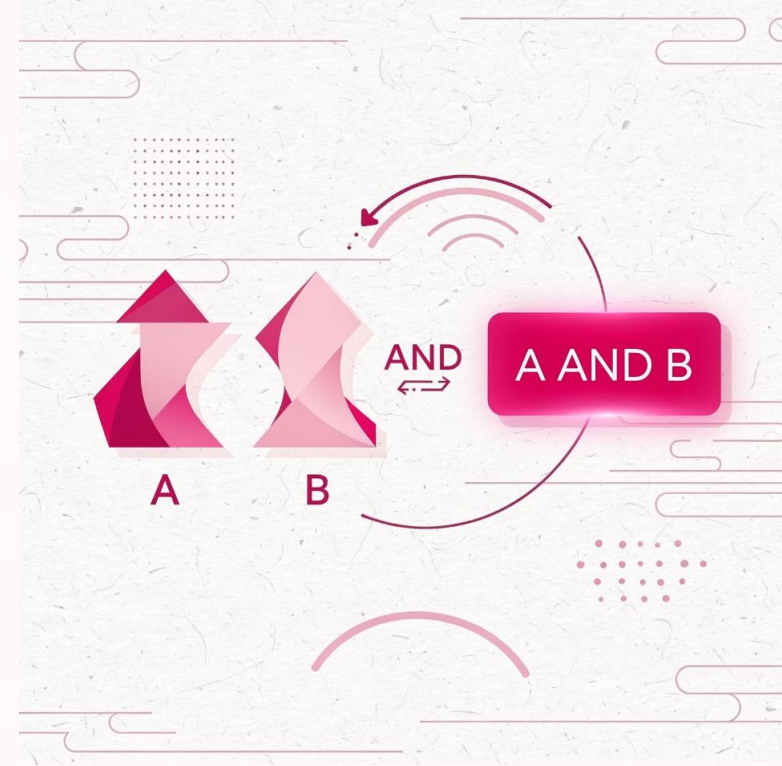
AND Kapısı: Mantıksal Çarpım

AND kapısı, en temel mantık kapılarından biridir ve mantıksal çarpım işlemini gerçekleştirir. AND kapısının çıkışı, **sadece ve sadece tüm girişleri 1 (TRUE) olduğunda** 1 olur. Aksi tüm durumlarda çıkış 0'dır.

Çalışma prensibi: AND kapısı, günlük hayatta "ve" bağlacına karşılık gelir. Örneğin, "Hem yağmur yağıyor VE şemsiyem var ise ıslak kalmam" ifadesinde olduğu gibi, her iki koşulun da sağlanması gerekir.

Matematiksel gösterim:

- Sembol: $A \wedge B$ veya $A \cdot B$
- Çıkış formülü: $Y = A \text{ AND } B$
- Boolean denklemi: $Y = A \cdot B$



Doğruluk Tablosu (Truth Table)

Giriş A	Giriş B	Çıkış Y
0	0	0
0	1	0
1	0	0
1	1	1

AND kapıları, güvenlik sistemlerinde, hesap makinelerinde ve veri filtreleme işlemlerinde yaygın olarak kullanılır.

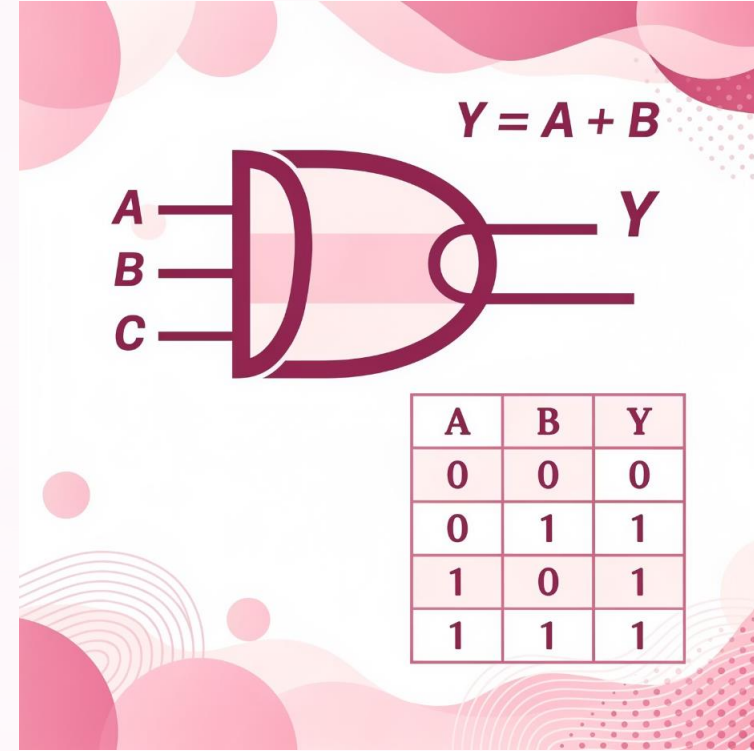
OR Kapısı: Mantıksal Toplam

OR kapısı, mantıksal toplama işlemini gerçekleştirir ve girişlerinden **en az birinin 1 (TRUE) olması durumunda** çıkışı 1 yapar. Çıkış sadece tüm girişler 0 olduğunda 0 değerini alır.

Çalışma prensibi: OR kapısı, günlük dilde "veya" bağlacına karşılık gelir. Örneğin, "Kredi kartımla VEYA nakit ile ödeyebilirim" ifadesinde olduğu gibi, seçeneklerden herhangi biri yeterlidir.

Matematiksel gösterim:

- Sembol: $A \vee B$ veya $A + B$
- Çıkış formülü: $Y = A \text{ OR } B$
- Boolean denklemi: $Y = A + B$



Doğruluk Tablosu (Truth Table)

Giriş A	Giriş B	Çıkış Y
0	0	0
0	1	1
1	0	1
1	1	1

OR kapıları, alarm sistemlerinde, çoklu sensör uygulamalarında ve karar verme devrelerinde sıklıkla kullanılır. Herhangi bir koşulun sağlanması yeterli olduğunda OR kapısı tercih edilir.

NOT Kapısı: Mantıksal Ters Alma

NOT kapısı, diğer adıyla inverter (ters çevirici), en basit mantık kapısıdır ve sadece **tek bir giriş** alır. Girişin mantıksal tersini çıkış olarak üretir: giriş 1 ise çıkış 0, giriş 0 ise çıkış 1 olur.

Çalışma prensibi: NOT kapısı, günlük dilde "değil" ifadesine karşılık gelir. Örneğin, "Bugün tatil DEĞİL ise çalışmalıyım" ifadesinde olduğu gibi, bir durumun tersini temsil eder.

Matematiksel gösterim:

- Sembol: $\bar{A}$ veya $\neg A$ veya A'
- Çıkış formülü: $Y = \text{NOT } A$
- Boolean denklemi: $Y = \bar{A}$

NOT kapısı, sinyal terslemede, bit manipülasyonunda ve Boolean ifadelerin sadeleştirilmesinde kritik rol oynar. Ayrıca NAND ve NOR gibi evrensel kapıların oluşturulmasında da temel bileşendir.

Doğruluk Tablosu

Giriş A	Çıkış Y
0	1
1	0



Önemli: NOT kapısı, diğer karmaşık kapıları (NAND, NOR) oluşturmak için temel bir bileşendir.

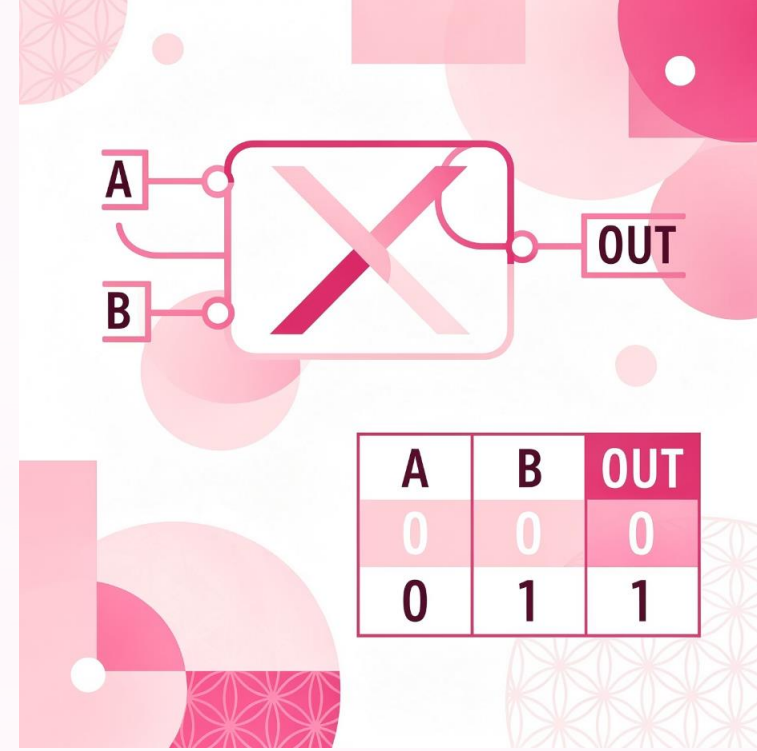
XOR Kapısı: Özel VEYA İşlemi

XOR (Exclusive OR - Özel VEYA) kapısı, girişleri **birbirinden farklı olduğunda** çıkışı 1 yapan özel bir mantık kapısıdır. Her iki giriş aynı değerde olduğunda (ikisi de 0 veya ikisi de 1) çıkış 0 olur.

Çalışma prensibi: XOR kapısı, "ya ... ya da ..." ifadesine karşılık gelir. Örneğin, "Ya kahve YA DA çay içerim, ikisini birden içmem" durumunda olduğu gibi, seçeneklerden sadece biri seçilmelidir.

Matematiksel gösterim:

- Sembol: $A \oplus B$
- Çıkış formülü: $Y = A \text{ XOR } B$
- Boolean denklemi: $Y = A \cdot \bar{B} + \bar{A} \cdot B$



Doğruluk Tablosu (Truth Table)

Giriş A	Giriş B	Çıkış Y
0	0	0
0	1	1
1	0	1
1	1	0

XOR kapısı, toplayıcı devrelerde, parite kontrol sistemlerinde ve kriptografi uygulamalarında yaygın olarak kullanılır. Özellikle veri şifreleme algoritmalarında temel bir role sahiptir.

Python ile Sanal Mantık Devresi Simülatörü

BLM101 dersi kapsamında geliştirilen bu proje, teorik mantık kapısı bilgisini pratiğe dökmeyi amaçlayan interaktif bir simülasyon uygulamasıdır. Python programlama dili kullanılarak tasarlanan simülatör, öğrencilerin mantık devrelerini görselleştirmelerine ve çeşitli girdi kombinasyonlarıyla denemeler yapmalarına olanak tanır.

01

Eğitsel Amaç

Mantık kapılarının çalışma prensiplerini somut bir uygulama ile pekiştirmek ve teorik bilgiyi yazılıma dönüştürme becerisini geliştirmek.

02

İnteraktif Öğrenme

Kullanıcıların farklı girdi değerleriyle denemeler yaparak öğrenmelerini sağlamak ve anlık geri bildirim sunmak.

03

Doğruluk Tablosu Üretimi

Her mantık kapısı için otomatik doğruluk tabloları oluşturarak tüm olası kombinasyonları görselleştirmek.

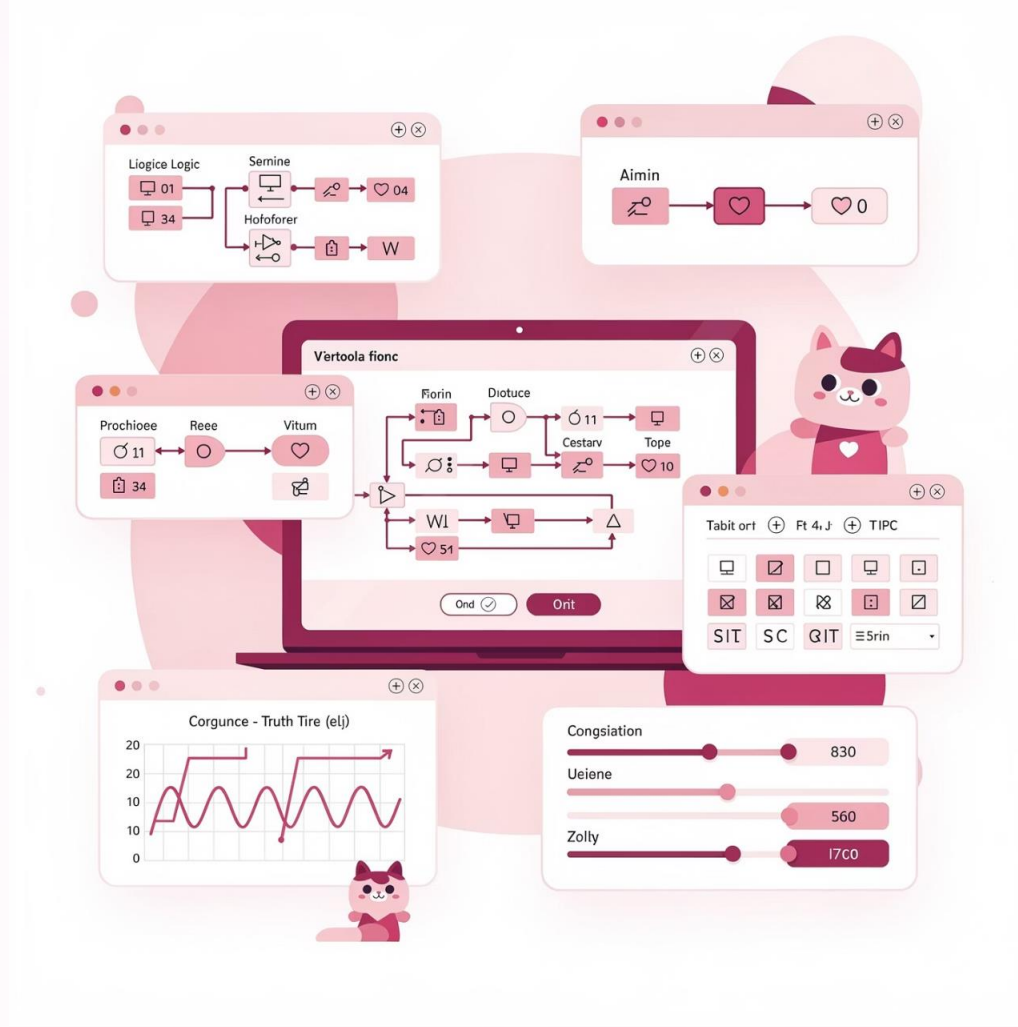
04

Karmaşık Devre Desteği

Birden fazla kapının birleştirildiği karmaşık mantık ifadelerini simüle edebilme yeteneği kazandırmak.

Bu proje, Python'un temel yapılarını (koşullu ifadeler, döngüler, fonksiyonlar) kullanarak gerçek dünya problemlerine yazılımsal çözümler üretme pratiği sağlar.

Projenin Amacı ve Kapsamı



Sanal Mantık Devresi Simülatörü projesi, bilgisayar mühendisliği öğrencilerine dijital mantık devrelerini anlama ve uygulama konusunda kapsamlı bir deneyim sunmayı hedefler.

Proje kapsamında gerçekleştirilen işlevler:

- *Temel mantık kapılarının (AND, OR, NOT, XOR) simülasyonu*
- *Kullanıcı tanımlı girdi değerleriyle hesaplama*
- *Otomatik doğruluk tablosu oluşturma*
- *Karmaşık mantık ifadelerinin değerlendirilmesi*
- *Adım adım işlem açıklamaları*

Akademik Hedefler

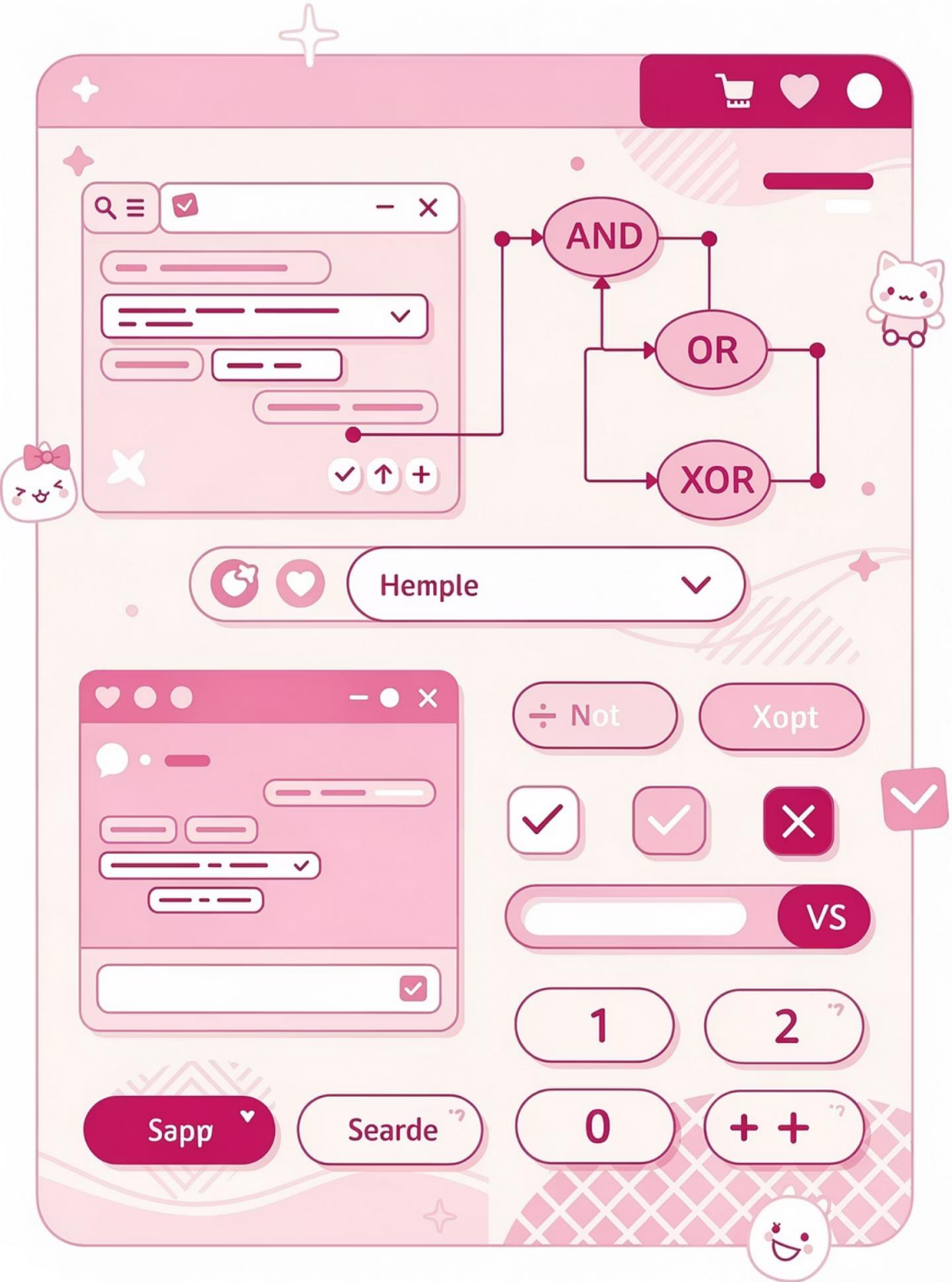
Boolean mantığı ve dijital devre tasarımının temel prensiplerine hakim olmak, teorik bilgiyi uygulamaya dönüştürmek.

Programlama Becerileri

Python programlama dilinde fonksiyon yazma, hata yönetimi ve kullanıcı etkileşimi konularında yetkinlik kazanmak.

Problem Çözme

Karmaşık mantıksal problemleri parçalara ayırarak sistematik çözümler üretme yeteneği geliştirmek.



KULLANIM

Programın Çalışma Mekanizması

Simülâtör, kullanıcı dostu bir komut satırı arayüzü aracılığıyla çalışır ve kullanıcılardan belirli girdiler alarak mantık işlemleri gerçekleştirir. Program, modüler bir yapıya sahiptir ve her mantık kapısı ayrı bir fonksiyon olarak tanımlanmıştır.



Kullanıcı Girdisi

Program başlatıldığında kullanıcıdan ikili değerler (0 veya 1) ve işlem türü seçimi istenir.



Mantık Kapısı Seçimi

Kullanıcı, AND, OR, NOT veya XOR kapılarından birini seçer veya karmaşık bir ifade girer.



İşlem Gerçekleştirme

Seçilen kapıya göre ilgili fonksiyon çağrılır ve Boolean mantık kuralları uygulanır.



Sonuç Gösterimi

Hesaplanan değer ekrana yazdırılır ve istenirse doğruluk tablosu oluşturulur.



Önemli Not: Program, hatalı girdileri tespit eder ve kullanıcıya uygun hata mesajları gösterir. Sadece 0 ve 1 değerleri kabul edilir.

AND

OR

XOR

SİMÜLASYON ADIM 1

Basit İki Girişli İşlemler

Simülasyonun ilk aşamasında, kullanıcı iki ikili değer (0 veya 1) girer ve bir mantık kapısı seçer. Program, bu girdileri alır ve seçilen kapının mantık kurallarına göre çıkışı hesaplar.

Örnek 1: AND İşlemi

Giriş: $A = 1, B = 0$

İşlem: $A \text{ AND } B$

Çıkış: 0

Açıklama: Her iki giriş de 1 olmadığı için AND kapısının çıkışı 0'dır.

Örnek 2: OR İşlemi

Giriş: $A = 1, B = 0$

İşlem: $A \text{ OR } B$

Çıkış: 1

Açıklama: Girişlerden en az biri 1 olduğu için OR kapısının çıkışı 1'dir.

Örnek 3: XOR İşlemi

Giriş: $A = 1, B = 1$

İşlem: $A \text{ XOR } B$

Çıkış: 0

Açıklama: Her iki giriş aynı değerde olduğu için XOR kapısının çıkışı 0'dır.

Bu basit işlemler, mantık kapılarının temel davranışını anlamak için kritik öneme sahiptir ve daha karmaşık devrelerin temelini oluşturur.

Doğruluk Tablosu Üretimi

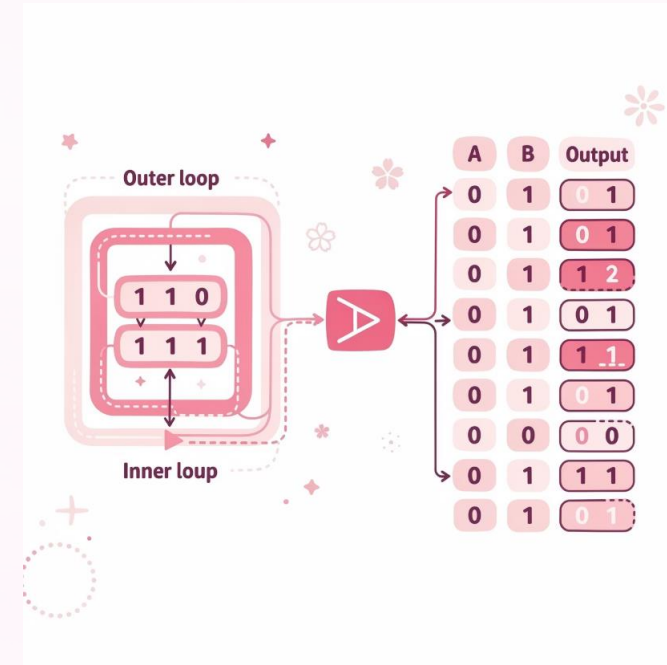
Programın en güçlü özelliklerinden biri, herhangi bir mantık kapısı için otomatik olarak doğruluk tablosu oluşturma yeteneğidir. Bu özellik, iç içe döngüler kullanılarak tüm olası girdi kombinasyonlarını sistematik olarak test eder.

Algoritma Mantığı

```
def generate_truth_table(gate_type):
    print("Doğruluk Tablosu:")
    print("A | B | Çıkış")
    print("-" * 15)

    for a in [0, 1]:
        for b in [0, 1]:
            if gate_type == "AND":
                output = and_gate(a, b)
            elif gate_type == "OR":
                output = or_gate(a, b)
            elif gate_type == "XOR":
                output = xor_gate(a, b)

            print(f"{a} | {b} | {output}")
```



Döngü Yapısı: İç içe iki for döngüsü kullanılarak $2^2 = 4$ farklı kombinasyon test edilir.

Bu özellik, öğrencilerin her mantık kapısının tüm olası durumlarını görmelerini sağlar ve teorik bilgilerin doğruluğunu pratik olarak teyit etmelerine yardımcı olur. Üç girişli kapılar için algoritma $2^3 = 8$ kombinasyonu test eder.

Sıkça Kullanılan Türetilmiş Kapılar

Temel mantık kapılarına ek olarak, dijital devre tasarımında sıkça kullanılan ve daha karmaşık işlevsellik sağlayan bazı türetilmiş kapılar bulunmaktadır. Bu kapılar, temel kapıların birleşiminden oluşur ve devreyi basitleştirmede, maliyeti düşürmede ve işlem gücünü artırmada önemli rol oynarlar.



NAND Kapısı

NAND (NOT-AND) kapısı, AND kapısının tam tersi şekilde çalışır. Tüm girişler 1 olduğunda çıkışı 0, diğer tüm durumlarda ise çıkışı 1'dir. Evrensel bir kapıdır, yani diğer tüm mantık kapıları NAND kapıları kullanılarak oluşturulabilir.

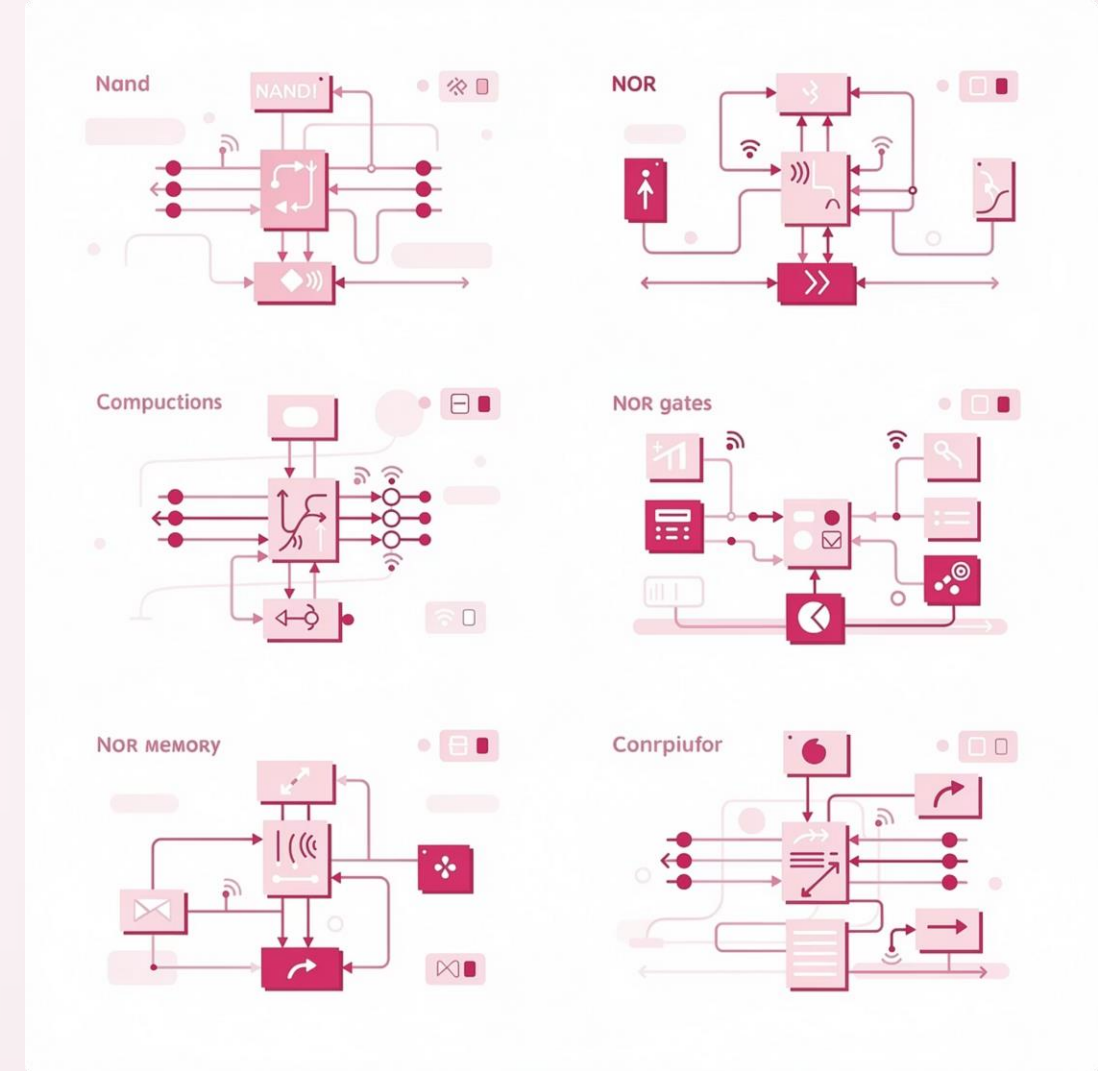


NOR Kapısı

NOR (NOT-OR) kapısı, OR kapısının tam tersi işlev görür. Tüm girişler 0 olduğunda çıkışı 1, diğer tüm durumlarda ise çıkışı 0'dır. NAND gibi, NOR kapısı da evrensel bir kapıdır ve dijital tasarımda temel yapı taşı olarak kullanılır.

Bu kapıların simülatörümüze eklenmesi, öğrencilere dijital mantık devrelerinin çeşitliliğini ve esnekliğini anlama fırsatı sunar.

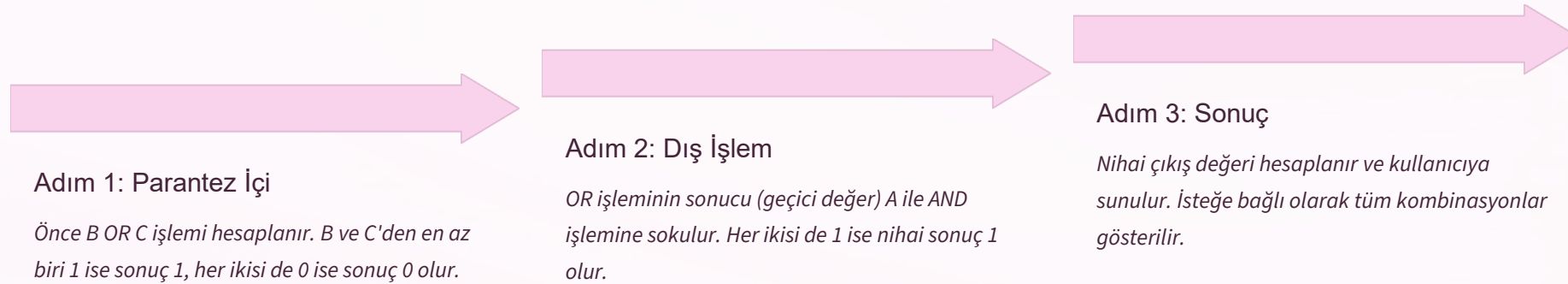
Özellikle, evrensellik kavramı bu kapıların önemini vurgular.



Karmaşık Örnek: A AND (B OR C)

Proje kapsamında özel olarak incelenen karmaşık mantık ifadesi, birden fazla kapının birleştirilmesiyle oluşturulan bir devredir. "A AND (B OR C)" ifadesi, önce iç parantezdeki OR işlemini, ardından sonucu A ile AND işlemine tabi tutar.

İfadenin Çözümlemesi



Tam Doğruluk Tablosu

A	B	C	B OR C	Sonuç
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	1	0
1	0	0	0	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

Program Çıktısı ve Ekran Görüntüleri

Bu bölümde, Python simülatörünün çalışma anındaki ekran görüntüleri ve örnek çıktılar sunulmaktadır. Kullanıcı arayüzü, komut satırı tabanlı olup basit ve anlaşılır tasarlanmıştır.



Örnek Program Çıktısı

```
=====
MANTIK DEVRESİ SİMÜLATÖRÜ
=====

Mevcut İşlemler:
1. AND Kapısı
2. OR Kapısı
3. NOT Kapısı
4. XOR Kapısı
5. Karmaşık İfade (A AND (B OR C))
6. Doğruluk Tablosu Üret

Seçiminiz (1-6): 5

A değerini girin (0 veya 1): 1
B değerini girin (0 veya 1): 0
C değerini girin (0 veya 1): 1

Hesaplama Adımları:
-----
B OR C = 0 OR 1 = 1
A AND (B OR C) = 1 AND 1 = 1

Sonuç: 1

Devam etmek istiyor musunuz? (E/H):
```

Sonuç ve Öğrenilen Kavramlar

Bu proje, bilgisayar mühendisliğinin temel taşlarından biri olan dijital mantık ve veri manipülasyonu konularını kapsamlı bir şekilde ele almıştır. Teorik bilgi ile pratik uygulama arasında köprü kurarak, öğrencilerin konuyu derinlemesine kavramalarını sağlamıştır.



Dijital Mantık Temelleri

Mantık kapılarının çalışma prensipleri, Boolean cebri ve dijital devrelerin temel yapıtaşları hakkında derinlemesine bilgi edinildi.



Python Programlama

Fonksiyonlar, döngüler, koşullu ifadeler ve hata yönetimi gibi temel programlama kavramları pratik olarak uygulandı.



Algoritma Tasarımı

Karmaşık problemleri parçalara ayırma, sistematik çözüm yolları geliştirme ve verimli algoritmalar yazma becerileri geliştirildi.



Doğrulama ve Test

Yazılan kodun doğruluğunu test etme, hata ayıklama ve doğruluk tablolarıyla sonuçları teyit etme yöntemleri öğrenildi.

Bu simülasyon projesi, BLM101 dersinin hedeflerine ulaşmada önemli bir araç olmuş ve teorik bilgilerin somut uygulamalara dönüştürülmesinde başarılı bir örnek teşkil etmiştir. Öğrenciler, bu çalışma sayesinde bilgisayar mimarisinin temellerini hem teorik hem de pratik düzeyde kavramış bulunmaktadır.

Kaynaklar ve Sorular

Kaynakça

1. *Digital Design and Computer Architecture*, David Harris & Sarah Harris, 2012
2. *Introduction to Logic Circuits & Logic Design with VHDL*, Brock J. LaMeres, 2019
3. *Python Crash Course: A Hands-On Project-Based Introduction to Programming*, Eric Matthes, 2019
4. *Computer Organization and Design*, Patterson & Hennessy, 5. Baskı
5. *Boolean Algebra and Its Applications*, J. Eldon Whitesitt, Dover Publications
6. *The Art of Computer Programming*, Donald E. Knuth, Cilt 1-4

Çevrimiçi Kaynaklar

- *Python.org resmi dokümantasyonu*
- *GeeksforGeeks - Digital Electronics dersleri*
- *Coursera - Computer Architecture kursu*



Sorularınız için

- Bu sunumla ilgili sorularınızı, projenin teknik detaylarını veya mantık devreleri hakkındaki merak ettiklerinizi sormaktan çekinmeyin.

İletişim Bilgileri

Ders Sayfası: [Bilgisayar Mühendisliğine Giriş](#)

GitHub Repo: [Nurr-E/BLM101_25360859042_Elif-in](#)

"Basit prensipler, karmaşık sistemlerin temelini oluşturur. Mantık kapıları, modern bilgisayarların en küçük ancak en kritik yapı taşlarıdır."

Teşekkürler!